

R - Control IP 4 – HTTP/JSON API

What is this?

The HTTP/JSON API is the direct line to the device. From your own software you can

- **switch** outputs and **read** their state,
- create and change **schedules** and **temperature actions**,
- read the **temperature history** and the **event log**,
- read and write **settings**,
- monitor the **device status**.

You use the same interface the device's web interface uses — everything you see there, you can fetch yourself.

Which interface is right for me?

You want to ...	Use
switch and configure from your own software	this API
integrate the device into Home Assistant	MQTT
connect a PLC or building management system	Modbus TCP
be notified when something happens	Webhook
follow state changes in real time	WebSocket

First switching command in 5 minutes

All examples use **curl**. Replace **192.168.0.3** with your device's address.

Step 1: Log in and get a token

```
curl -sS -X POST "http://192.168.0.3/api/auth/login" \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"YourPassword"}'
```

Response:

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "token_type": "Bearer",  
  "expires_in": 1800,  
  "user": { "name": "admin", "permissions":  
    ["controls","overview","channels","schedules","actions","settings","about"] }  
}
```

You need the value of **token** for every further call.

Step 2: Keep the token

```
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Step 3: Switch output 1 on

```
curl -sS -X POST "http://192.168.0.3/api/relay" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"index":0,"command":"on"}
```

Step 4: Read the state

```
curl -sS "http://192.168.0.3/api/system/status" \
-H "Authorization: Bearer $TOKEN"
```

That's it. Every other call follows the same pattern: **URL + Authorization header**, plus JSON in the body for writing calls.

Basics

Address

```
http://<device>:<port>/api/...
```

The port is shown in the device settings under **Network** → **Web server** (default **80**). With HTTPS enabled, use **https://** on the same port.

Instead of the IP address the hostname usually works too, e.g. **http://r-control-ip4.local**.

Authentication

Every call except the login needs the header:

```
Authorization: Bearer <token>
```

If the token is missing or expired, the device answers **401** with **{"error":"unauthorized"}**. Get a new token through the login endpoint.

Lifetime: 30 minutes by default. At login you can request a different duration with **"ttl": 3600** (60 to 86400 seconds). For long-running services the simplest approach is to log in again automatically whenever a call returns 401.

Protection against password guessing

After **10 failed login attempts** the login is **blocked for 15 minutes** — for everyone, device-wide. The response is then:

```
HTTP 429 Too Many Requests
Retry-After: 873
{"error":"locked","retry_after_s":873}
```

Check your credentials carefully while developing; a loop with a wrong password locks you and everyone else out. The current state can be queried without authentication:

```
curl -sS "http://192.168.0.3/api/auth/status"
# {"locked":false,"retry_after_s":0,"attempts_left":10}
```

Permissions

Every user has permissions that determine which endpoints they may use:

Permission	Allows
controls	switching outputs
overview	temperature history, event log, device status
channels	outputs and their settings
schedules	schedules
actions	temperature actions
settings	configuration, users, firmware, restart
about	device information

If a permission is missing, the device answers **403** and names the required permission. For your own software, create a dedicated user that only has the permissions it really needs.

Responses and errors

Success is always JSON with HTTP 200:

```
{"status":"ok"}
```

Errors look like this:

```
{"status": "error", "code": "invalid_request", "message": "Invalid input"}
```

HTTP code	Meaning	What to do
400	Malformed request	Check JSON and field names
401	Not authenticated	Get a new token
403	Missing permission or wrong password	Check the user's permissions
404	Not found	Check the URL
409	Conflict	e.g. user exists, limit reached
413	Too large	File or payload limit exceeded
429	Too many attempts	Login lockout, wait <code>retry_after_s</code>
500	Device error	Read the message; export diagnostics if needed

Switching

Switch an output

```
POST /api/relay
```

```
{"index": 0, "command": "on"}
```

`index` is **zero-based**: 0 = output 1 ... 3 = output 4.

command	Effect
on	switch on
off	switch off
toggle	toggle
impulse	switch briefly, then back (see below)

Impulse — for example a door opener for 3 seconds:

```
{"index": 0, "command": "impulse", "state": "on", "duration_ms": 3000}
```

After the time has elapsed the output falls back to the opposite state automatically.

Example: switch all four outputs on, one after another

```
for i in 0 1 2 3; do
  curl -sS -X POST "http://192.168.0.3/api/relay" \
    -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
    -d "{\"index\":\"$i\",\"command\":\"on\"}"
done
```

Requires permission: controls

State and monitoring

Read the overall status

```
GET /api/system/status
```

The single most useful call: in one request it returns firmware version, uptime, time status, temperature, the state of all interfaces, schedule counters and security notices.

```
{
  "system": { "app_version": "1.4.0", "uptime_s": 86400, "reset_reason":
    "poweron",
    "reset_unexpected": false, "heap_free": 118000, "hostname": "R-
Control-IP4" },
  "time": { "plausible": true, "synced": true, "last_sync_source": "sntp",
    "rtc_ok": true },
  "sensor": { "ntc_ok": true, "temperature_c": 21.4 },
  "interfaces": {
    "mqtt": { "enabled": true, "connected": true },
    "modbus": { "enabled": false, "running": false, "port": 502 },
    "webhook": { "enabled": true, "configured": true, "sent_ok": 42,
    "sent_failed": 0 }
  },
  "automation": { "scheduler_enabled": true, "schedule_count": 3,
    "schedule_limit": 30 },
  "security": { "login_lockout_remaining_s": 0, "admin_uses_default_password":
    false }
}
```

For monitoring it is usually enough to fetch this one object every 30–60 seconds.

Requires permission: overview

Network status

```
GET /api/network/status
```

Returns link state, IP address, gateway, DNS, MAC and whether the address came from DHCP or is static.

Temperature history

```
GET /api/telemetry/temperature
```

```
{
  "enabled": true,
  "intervalSeconds": 60,
  "capacity": 1440,
  "samples": [ {"t": 1789720000000, "v": 21.4}, {"t": 1789720060000, "v": 21.5} ]
}
```

t is the timestamp in milliseconds, **v** the temperature in °C. The device keeps up to 1,440 samples; at a 60-second interval that equals 24 hours.

Event log

```
GET /api/overview/actions
```

The last 32 events (switching operations, button presses, system events), newest first.

```
{
  "enabled": true,
  "entries": [
    {"timestampMs": 1789720000000, "type": "output", "channel": 2,
     "name": "Yard light", "action": "on", "actionText": "Switched on"}
  ]
}
```

Requires permission: [overview](#)

Schedules

Read all

```
GET /api/scheduler
```

Write all

```
POST /api/scheduler
```

Important: The call replaces **all** schedules. Read first, modify the result, then write it back completely. Individual entries cannot be created or deleted separately.

Example: a daily switch at 07:00

```
{
  "enabled": true,
  "rules": [
    {
      "id": 17897200000000,
      "name": "MorningOn",
      "type": "daily",
      "time": "07:00:00",
      "time_mode": "fixed",
      "enabled": true,
      "per": [[ "on", "state", 0 ], [ "hold", "state", 0 ], [ "hold", "state", 0 ],
[ "hold", "state", 0 ] ]
    }
  ]
}
```

The **per** field describes what each of the four outputs should do — one entry per output:

[action, mode, impulse duration in ms]

Position	Possible values
action	"on" switch on · "off" switch off · "hold" leave unchanged
mode	"state" permanent · "impulse" briefly
impulse duration	milliseconds, only with "impulse"

Recurrence (**type**):

Value	Meaning	Additional fields
once	one time	year, month, day
daily	daily	–
weekly	weekly	week_days: [1,2,3,4,5] (0 = Sunday)
monthly	monthly	day
yearly	yearly	month, day

Sunrise/sunset instead of a fixed time:

```
{
  "id": 1789720000001,
  "name": "EveningOn",
  "type": "daily",
  "time_mode": "sunset",
  "astro_offset_minutes": -15,
  "astro_not_before": "17:30",
  "astro_not_after": "21:00",
  "enabled": true,
  "per": [[ "on", "state", 0 ], [ "hold", "state", 0 ], [ "hold", "state", 0 ],
  [ "hold", "state", 0 ] ]
}
```

This switches 15 minutes **before** sunset, but not earlier than 17:30 and not later than 21:00. A location must be configured under **Settings** → **General** → **Location**.

Preview: when will a rule run next?

```
GET /api/scheduler/next-runs?id=<rule id>&count=5
```

```
{
  "time_plausible": true,
  "scheduler_enabled": true,
  "runs": [
    { "epoch": 1789750800, "status": "ok", "astro": true, "astro_limited": false },
    { "epoch": 1789837200, "status": "skip_holiday", "astro": true,
    "astro_limited": true }
  ]
}
```

The device uses the same logic as during operation. Dates skipped because of holiday or vacation rules are included with a **status** — handy for checking a rule before putting it to work.

Requires permission: `schedules`

Temperature actions

```
GET /api/actions
POST /api/actions
```

Here too, the **POST** replaces **all** rules.

Example: switch a fan on above 25 °C, off below 23 °C

```
{
  "enabled": true,
  "rules": [
    {
      "id": 1789720000002,
      "name": "Fan",
      "enabled": true,
      "condition": { "operator": "above", "threshold": 25.0, "clear_threshold":
23.0 },
      "cooldown_seconds": 60,
      "trigger": { "per": { "ch1": { "act": "on", "mode": "state" } } },
      "recover": { "per": { "ch1": { "act": "off", "mode": "state" } } }
    }
  ]
}
```

Field	Meaning
operator	above = above the threshold, below = below it
threshold	Trigger temperature — the action runs here
clear_threshold	Release temperature — only when the temperature returns here can the action trigger again
cooldown_seconds	Lockout time after triggering
trigger	What happens when triggering
recover	What happens on release (optional)

The release temperature prevents constant switching back and forth with a fluctuating temperature. Leave it out and the action is released as soon as the temperature crosses back over the trigger threshold — with a value close to the threshold that can happen very often.

Requires permission: actions

Reading and writing settings

Full configuration

```
GET /api/config
POST /api/config
```

The **GET** returns the complete device configuration as JSON, the **POST** replaces it.

Careful: This is all-or-nothing as well. Read the configuration, change individual values, then write the whole thing back. **Never** write a hand-crafted partial JSON — missing sections are lost.

Example: change only the MQTT broker (using `jq`)

```
curl -sS "http://192.168.0.3/api/config" -H "Authorization: Bearer $TOKEN" \
  | jq '.config.interfaces.ha_mqtt.uri = "mqtt://192.168.0.50:1883"' \
  > new.json

curl -sS -X POST "http://192.168.0.3/api/config" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  --data-binary @new.json
```

Backup as a file

```
GET /api/config/bundle      → download TAR archive
POST /api/config/bundle     → import TAR archive
```

The archive contains `config.json`, `scheduler.json` and `actions.json`. Ideal for saving a device state or transferring it to a second device.

```
# Backup
curl -sS "http://192.168.0.3/api/config/bundle" \
  -H "Authorization: Bearer $TOKEN" -o backup.tar

# Restore
curl -sS -X POST "http://192.168.0.3/api/config/bundle" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/x-tar" \
  --data-binary @backup.tar
```

If the bundle contains network changes, the device restarts and is then reachable at the new address.

Apply network settings

After changing network values in the configuration:

```
POST /api/network/apply
```

Only then do the new settings take effect.

Requires permission: `settings`

Other useful calls

Call	Purpose
GET /api/about/app	Firmware and ESP-IDF version
POST /api/system/restart	Restart the device
GET /api/system/diagnostics	Diagnostics archive (TAR) for support
POST /api/interfaces/mqtt/test	Test the MQTT broker connection
POST /api/interfaces/webhook/test	Test the webhook target
POST /api/network/wan-refresh	Re-detect the WAN IP
DELETE /api/telemetry/temperature	Delete the temperature history
DELETE /api/overview/actions	Delete the event log

User management (GET/POST/PUT/DELETE /api/users) and firmware update (POST /api/system/ota, GET/POST /api/system/rollback) are available as well; the web interface covers both completely.

A complete example

A small Python script that logs in, reads the temperature and switches an output on above 25 °C:

```
import requests

DEVICE = "http://192.168.0.3"
USERNAME = "integration"
PASSWORD = "..."

# 1. Log in
resp = requests.post(f"{DEVICE}/api/auth/login",
                    json={"username": USERNAME, "password": PASSWORD}, timeout=5)
resp.raise_for_status()
token = resp.json()["token"]
head = {"Authorization": f"Bearer {token}"}

# 2. Read status
status = requests.get(f"{DEVICE}/api/system/status", headers=head,
                     timeout=5).json()
temperature = status.get("sensor", {}).get("temperature_c")
print(f"Temperature: {temperature} °C")

# 3. Switch if needed
if temperature is not None and temperature > 25.0:
    requests.post(f"{DEVICE}/api/relay", headers=head,
                json={"index": 0, "command": "on"}, timeout=5)
    print("Output 1 switched on")
```

For scripts like this, use a **dedicated user** with only the permissions required (here: [overview](#) and [controls](#)) — not the admin account.

Troubleshooting

"unauthorized" on every call

Is the **Authorization: Bearer <token>** header missing, or has the token expired? Tokens are valid for 30 minutes by default. Also check that you did not copy quotation marks into the token.

HTTP 429 on login

The login lockout is active — 10 failed attempts. **retry_after_s** in the response gives the remaining time. Restarting the device also clears the lockout.

Configuration changes disappear

You probably wrote a partial JSON. **POST /api/config** replaces the **entire** configuration — always read first, then modify, then write back completely.

A schedule does not run

1. Is the clock set? **GET /api/system/status** → **time.plausible** and **time.synced**
2. Is the scheduler globally enabled? → **automation.scheduler_enabled**
3. What does the preview say? **GET /api/scheduler/next-runs?id=...** also shows skipped dates with the reason.

The device does not answer at all

Check reachability and port (**ping**, correct HTTP port in the settings). With HTTPS enabled the device answers **only** via **https://** on the same port.

Notes for production use

- **A dedicated user per integration**, with minimal permissions. The event log then also shows which system triggered something.
- **Reuse the token**, do not log in before every call. On 401, log in once more.
- **Do not poll too often**. For ongoing monitoring, **GET /api/system/status** every 30–60 seconds is enough. If you need changes in real time, use the WebSocket interface.
- **Use HTTPS on open networks**. Without HTTPS, password and token travel in the clear. On an isolated plant network HTTP is acceptable, beyond that it is not.
- **Plan for timeouts**. The device is a small controller; use timeouts of a few seconds and handle failures instead of blocking calls indefinitely.